Box2D v3 原理与实现

从 Erin Catto 的公开文章，走到可以一步一步手算的 Soft Step 求解器

整理日期：2026-09-28。对象是会写程序、但希望把物理模拟算明白的人。公式全部用下标和普通符号写出，不依赖 LaTeX 预览。

> **关于原文。** Erin Catto 的 GDC 讲稿和 box2d.org 博客有版权，这份材料**不是**把那些 PDF 和博文整篇翻译粘贴进来。下面是按他公开过的算法，用中文重新写的讲解：每条公式都写出它从哪一步来，每个关键过程都配了可以手算的数字。每章开头标明对应的原始文章，方便你对照。Box2D 源码本身是 MIT 许可，文中引用的求解步骤与 `b2MakeSoft`、`solver.c`、`contact_solver.c` 一致（以 2024–2025 年 v3 主线为准）。

0. 文章地图：v3 站在哪些文章上

Box2D 的作者是 Erin Catto。2.x 是 C++ 的顺序冲量引擎；3.0（2024 年 8 月发布）是用 C 重写的，求解器换成他称为 **Soft Step** 的方案：软约束 + 子步，不再让用户填“速度迭代次数 / 位置迭代次数”。

0.1 仍然是 v3 数学基础的四次演讲

文章	年份	v3 里还在用的思想
Iterative Dynamics with Temporal Coherence box2d.org/files/ErinCatto_IterativeDynamics_GDC2005.pdf	GDC 2005	投影高斯–赛德尔（PGS）。冲量要累积再夹紧。上一帧的冲量当作下一帧的初值（热启动 / 时间相干）。接触点要能对上号。
Modeling and Solving Constraints ErinCatto_ModelingAndSolvingConstraints_GDC2009.pdf	GDC 2009	把 PGS 讲成更好写的“顺序冲量”。雅可比、有效质量、关节和接触是同一套公式。
Soft Constraints ErinCatto_SoftConstraints_GDC2011.pdf	GDC 2011	软约束不是先加一个弹簧力再硬解。它是改写后的速度约束，并且可以和隐式欧拉的弹簧–阻尼解成同一个结果，所以不会像显式弹簧那样炸。
Understanding Constraints ErinCatto_UnderstandingConstraints_GDC2014.pdf	GDC 2014	迭代不够时误差长什么样。块求解（一次解两个接触点）为什么堆箱子更稳。位置求解和速度求解要分开。

0.2 直接催生 v3 的博客

文章	日期	要记住的决定
Starting Box2D 3.0	2023–01	用推测性碰撞（speculative）代替“算撞击时刻再把整步切开”的串行 CCD。语言改成 C。线程由游戏自己的任务系统提供，库不私自开线程。
Simulation Islands	2023–10	岛是“物体为点、接触和关节为边”的连通块。静态地面不属于岛，否则所有东西连成一块。v3 用持久岛：大多数接触边跨帧不变，只处理新增和删除。休眠必须整岛一起睡。
Solver2D	2024–02	他做了一个砍掉睡眠、多线程、CCD 的试验引擎，并列比较 8 种求解器。最后选中的 TGS_Soft，发布时改名为 Soft Step。
Releasing Box2D 3.0	2024–08	Soft Step 在质量比、长链、大堆上比 2.4 稳。竖直叠箱子略逊于 2.4 的 NGS 块求解，他认为不值得为这一种场景付那个成本。接触求解用图着色 + AVX2。
SIMD Matters	2024–08	同一颜色的接触不共享刚体，所以可以并排解。物体状态在二维里刚好约 32 字节，适合 256 位向量。颜色跨帧保留，不必每帧重涂。
Determinism	2024–08	多线程不能用“谁先做完谁写入”的原子自增来决定接触顺序。各线程写自己的位图，主线程按位或合并，顺序就确定了。关掉 fast-math 和 FMA，atan2 用自己的近似，才能跨 CPU 得到同一结果。v3 没有回滚确定性。

更早的博文（鬼碰撞、预测关节限位、动态树）解释的是碰撞几何和宽相，不改变求解器公式。2025 年以后的 Box2D 3.1、动态树改进，以及 2026 年的 Box3D，是同一套求解思想往前延伸。这份讲解以 Soft Step 为准。

1. 先把符号钉死

后面所有例子只用这些字母。二维，逆时针为正，y 向上。

符号	是什么	怎么用
m，1/m	质量，逆质量	静态物体 1/m = 0，冲量推不动它
I，1/I	转动惯量，逆惯量	锁旋转时 1/I = 0
v，ω	质心速度，角速度	ω 是一个数，不是向量
P	接触点或锚点	冲量打在这里
r	P − 质心	力臂。从质心指向作用点
n	单位法线，从 A 指向 B	B 吃 +j n，A 吃 −j n
vn	$(vB^P - vA^P) \cdot n$	负数表示正在靠近
s 或 C	间隙。s > 0 分开，s < 0 嵌进去	v3 用这个符号，和“穿透深度为正”的旧习惯相反
h	这一小步的时间	子步时 h = 整帧时间 / 子步数
λ 或 j	累积冲量	接触法向要求 λ ⩾ 0
m_{eff}	有效质量	源码里叫 normalMass。等于 1/K
K	单位冲量能改变多少 vn	$K = 1/mA + 1/mB + (rA \times n)^2/IA + (rB \times n)^2/IB$

$$\text{点 P 的速度：} v^P = v + \omega \times r$$
$$\text{二维：} \omega \times r = (-\omega\, r_y,\ \omega\, r_x)$$
$$\text{叉积标量：} r \times n = r_x\, n_y - r_y\, n_x$$

K 的四项各自的意思：前两项是“质心被推开”，后两项是“转起来之后接触点还会再让出一点”。正撞圆心时叉积为 0，只剩 1/m。墙的 1/m 和 1/I 都是 0。

2. 顺序冲量：一次只修一个约束

2005 年的文章把这件事写成解线性方程组的投影高斯–赛德尔。2009 年他改口说：别先写大矩阵，对每个约束算一个冲量，立刻改两边速度，再看下一个。这就是顺序冲量。数学上仍是 PGS，只是按接触对来写，二维游戏里更好实现。Box2D 从第一版到 v3 都是这个循环，变的是“这一记冲量的公式”和“循环包在迭代里还是包在子步里”。

2.1 单个硬接触，一步就精确

硬接触、不弹（恢复系数 e = 0）时，目标是把 vn 修到 0，而且冲量不能是负的（只能推开）。

$$j = \max(\ 0,\ \lambda_{旧} + \Delta j\ ) - \lambda_{旧} \text{ 里先算}$$
$$\Delta j = -\, m_{eff} \cdot vn$$
$$\lambda_{新} = \max(0,\ \lambda_{旧} + \Delta j)$$
$$\text{真正加到速度上的是 } \lambda_{新} - \lambda_{旧}$$

为什么不直接用 $\Delta j = -m_{eff}\, vn$，而要绕过累积值？因为接触是单向的。如果这一次算出的增量是负的，它表示“之前推太狠了，要收回一点”。收回之后累积值如果穿过 0，就必须停在 0。只夹紧增量、不夹紧总和，物体会在该分开时仍被粘住，看起来就是抖。

> **例子 A：球撞墙，一个约束，一次就算完。**
> 墙是 A，静态。球是 B，m = 2，所以 $m_{eff} = 2$，K = 1/2 = 0.5。法线从墙指向球，球正在以 4 m/s 撞向墙，所以 vn = −4。λ 从 0 开始，e = 0。
> $\Delta j = -2 \times (-4) = 8$。$\lambda_{新} = 8$。
> 球的速度变化：$\Delta v = j / m = 8 / 2 = 4$，沿法线。原来 −4，加上 4，变成 0。
> 验算：vn 从 −4 变成 0。动量：墙不动，球的动量变化等于冲量 8，一致。
> 若 e = 0.5，公式多一个因子：$\Delta j = -m_{eff}\,(1+e)\, vn = -2 \times 1.5 \times (-4) = 12$。速度变成 −4 + 12/2 = +2。分开速度 2，正好是接近速度 4 的一半。v3 不把弹性放进这个主循环，而是所有子步结束之后单独做一次恢复，见第 8 章。

2.2 两个约束串在一起：为什么要来回修

一个约束时，一次高斯–赛德尔就是精确解。两个约束抢同一个物体时，修第二个会把第一个刚修好的速度弄坏，所以要重复。

> **例子 B：墙、箱子 1、箱子 2，都在一条直线上。**
> 两个箱子质量都是 1。墙在左边，静态。法线都指向右，vn > 0 表示在分开。
> 开始：v1 = −3，v2 = −1。两只箱子都在往左冲，1 号更快，会先顶到墙，2 号会顶到 1 号。
> 约束 G：墙和 1 号。$m_{eff} = 1$。$\Delta\lambda_G = -v1$，然后夹到 ⩾ 0，1 号速度加上这份冲量。
> 约束 H：1 号和 2 号。K = 1+1 = 2，$m_{eff} = 1/2$。$\Delta\lambda_H = -(v2-v1)/2$。1 号减去它，2 号加上它。
> 两个约束都从 λ = 0 开始。每一“轮”先做 G 再做 H：

轮	v1	v2	λ_G	λ_H
0	−3	−1	0	0
1	−0.5	−0.5	3	0.5
2	−0.25	−0.25	3.5	0.75
3	−0.125	−0.125	3.75	0.875
4	−0.0625	−0.0625	3.875	0.9375
6	−0.0156	−0.0156	3.9688	0.9844

> 第一轮之后两个速度已经相等，这是 H 的功劳。但它们还一起往左，G 还没完全把它们撑住。每一轮剩下的靠近速度减半。4 轮之后还剩 0.0625 m/s，6 轮接近停住，但有限次数永远差一点点。这就是“迭代次数不够时，堆叠会慢慢陷进去”的一维版本。

请对着表看第 1 轮的算术，确认没有跳步：

1. G：vn = v1 = −3。$\Delta j = -1 \times (-3) = 3$。$\lambda_G = 3$。v1 = −3+3 = 0。

2. H：vn = v2−v1 = −1−0 = −1。Δj = −(−1)/2 = 0.5。λ_H = 0.5。v1 = 0−0.5 = −0.5，v2 = −1+0.5 = −0.5。
3. 第 2 轮 G：vn = −0.5，Δj = 0.5，λ_G = 3.5，v1 = 0。然后 H 再把两个速度拉成 −0.25。和表一致。

2.3 热启动：把上一帧的 λ 先打回去

物体快停下来时，正确的支撑冲量其实每帧都差不多，大约等于“这一帧重力造成的动量变化”。如果每帧都从 λ = 0 开始，前几轮箱子会先掉下去再被推回来，画面就在抖。2005 年的办法：下一帧开始时，先把上一帧存下的 λ 施加到速度上，再迭代。这叫热启动，也叫时间相干。

> **例子 C：同一条链，每帧额外得到 Δv = −1（把重力想成每帧一次速度增量），只迭代 2 轮。**
> 冷启动：每帧把 λ 清零。
> 热启动：每帧先把旧 λ 打进速度，再迭代，λ 留下来。
>
> | 帧 | 冷启动 v1=v2 | 冷启动 λ_G | 热启动 v1=v2 | 热启动 λ_G |
> |---|---|---|---|---|
> | 1 | −0.25 | 1.5 | −0.25 | 1.5 |
> | 2 | −0.3125 | 1.875 | −0.125 | 2.25 |
> | 3 | −0.328 | 1.969 | 0 | 2.25 |
> | 4 | −0.332 | 1.992 | +0.031 | 2.06 |
>
> 冷启动永远停在大约 −0.33：2 轮修不完“这一帧新掉下来的 1 m/s”，残差每帧都在，箱子一直在陷。热启动到第 3 帧速度已经是 0，支撑冲量也稳定在 2.25 附近。第 4 帧多出来 +0.031，是累积冲量略大、把箱子推离墙一点点。v3 用“松弛趟”把这种多余的速度吃掉，而且松弛趟**不再改位置**。见第 6 章。

热启动能工作，前提是这一帧的接触和上一帧是“同一个”。所以接触要有稳定的特征编号（哪条边、哪个顶点）。点如果每帧换身份，热启动会把冲量打到错误的点上，比不热启动更抖。

2.4 质量差很大时，同样的轮数更不够

> **例子 D：地上是 1 kg 的轻箱子，上面压着 10 kg 的重箱子。每帧重力给两者 Δv = −1。只做 2 轮，冷启动。**
> 5 帧之后两者速度大约是 −2.93，还在加速下陷。支撑冲量在涨，但追不上“重箱子每帧需要大约 10 的冲量、地面还要再扛轻箱子”这件事。2.4 用更多速度迭代，外加一次解两个接触点的块求解，专门救竖直堆叠。v3 放弃块求解，改成把时间切成更小的子步。子步不是魔法：它让每小步的非线性更小，也让热启动在一帧内部多传递几次。质量比仍然不要随便放到几百比一。

3. 有效质量从“试推一下”推出来

2011 年软约束演讲里有一个不写雅可比也能懂的算法。约束冲量 λ 沿轴线推两端，推完去量两端锚点沿轴线的相对速度变化。这个变化量除以 λ，就是 K；倒数就是有效质量。

> **例子 E：水平冲量打在箱子角上。**
> 箱子 m = 2，I = 4/3，所以 1/m = 0.5，1/I = 0.75。质心在原点，接触点 r = (1, 1)，法线 n = (1, 0)，墙在右侧且不动。
> 1. 平移：单位冲量让质心速度增加 0.5，沿 n 的分量是 0.5。
> 2. 力臂 r×n = 1×0 − 1×1 = −1。
> 3. 单位冲量让角速度增加 1/I × (r×n) = 0.75×(−1) = −0.75。这里符号按“B 受 +j n”来；墙是 B 或 A 要和引擎约定一致，平方之后相同。
> 4. 转动在接触点造成的速度是 ω×r。它再点上 n，贡献 $(r\times n)^2 / I = 0.75$。
> 5. 墙贡献 0。所以 K = 0.5 + 0.75 = 1.25，$m_{eff} = 0.8$。
> 若不准转（1/I = 0），K 只有 0.5，同样的靠近速度需要更大的冲量，质心会被推得更狠。允许转之后，一部分效果变成旋转，法向冲量变小。这就是擦边、角撞墙会转的原因。两圆的法线一定过圆心，r×n = 0，法向冲量本身不产生转动，球要转只能靠摩擦。

4. 位置误差：Baumgarte、NGS，然后才是软约束

只把 vn 修成 0，只能保证“下一瞬间不再靠近”。如果这一帧开始时已经嵌进地面（s < 0），重叠会留在那里。历史上 Box2D 用过三种补法。

4.1 Baumgarte：在速度目标里掺一点位置误差

$$\Delta j = -\, m_{eff}\,(\, vn + \beta\, s \,/\, h\,)$$

β 常取 0.1 到 0.2。s 为负时，s/h 是负的，括号里比 vn 更负，冲量更大，物体被多推开一点。β = 1 表示企图在这一步里把重叠全部消掉，容易推过了头，能量凭空增加，轻箱子会弹飞。β = 0 就是完全不管重叠。这是便宜的，但是调参，而且和“弹性”混在同一个冲量里。

4.2 NGS：速度解完之后，用假速度去挪位置

Box2D 1.0 起用非线性高斯–赛德尔做位置修正。形式很像把 Baumgarte 里的 vn 和 h 拿掉：

$$\text{位置冲量} = -\, m_{eff} \cdot \beta \cdot \min(0,\, s)$$

这个冲量加到位置和转角上，不加到真正的 v 和 ω 上。所以它几乎不制造动能。平衡时位置冲量应该是 0，因此不要热启动、不要累积，否则接触会把已经分开的物体拉回来。2.4 的竖直堆叠很稳，很大程度是“速度迭代 + 位置迭代 + 两个接触点一起解”。代价是每帧要多扫好几遍约束。Solver2D 的对比里，Erin 认为这笔钱不如花在子步上。

4.3 显式弹簧为什么会炸

先看一个一维弹簧，这样后面的软约束才有对照。质量 m 连着弹簧 k 和阻尼 c：

$$m\, a = -c\, v - k\, x$$

改用两个更好调的数。角频率 $\omega = \sqrt{(k/m)}$，阻尼比 $\zeta = c / (2\sqrt{(k\ m)})$。$\zeta < 1$ 会振荡，$\zeta = 1$ 刚好不振荡（临界阻尼），$\zeta > 1$ 更黏、回得更慢。频率 f（赫兹）和 ω 的关系是 $\omega = 2\pi f$。k 和 c 的单位对人很不友好，而且真正“弹簧看见的质量”往往不是 m，而是有效质量，所以设计师应该调 f 和 ζ，而不是 k 和 c。

显式欧拉是先用当前速度改位置，再用当前加速度改速度。零阻尼时它每步都把能量推高，弹簧越硬炸得越快。半隐式欧拉先改速度再用新速度改位置，温和的弹簧能守住能量，但频率太高、步长太大时仍会炸。经验是一个振荡周期里至少走 4 步。隐式欧拉用**这一步结束时**的 v 和 x 来算加速度，任意硬都不会炸，只是步长越大能量被吃掉越多。刚性体求解器不想为每个弹簧解一次大型隐式方程。软约束的卖点是：仍用半隐式欧拉和顺序冲量，结果却和“这个弹簧用隐式欧拉走一步”相同。

5. 软约束三个系数：从隐式欧拉推到 b2MakeSoft

这是 v3 和 2011 年演讲之间最重要的一步。2011 年的 β、γ 还和有效质量缠在一起。Bepu 的作者 Ross Nordby 把它们收成三个只依赖 f、ζ、h 的数。Box2D v3 的 `b2MakeSoft` 就是这个形式。源码注释写明：massScale + impulseScale = 1。

5.1 隐式欧拉先走一步

沿约束坐标 x（这里先把 x 当成弹簧拉伸，后面再对应到间隙 s）。隐式欧拉：

$$v' = v + h\,a'$$
$$x' = x + h\,v'$$
$$m\,a' = -c\,v' - k\,x'$$

把 a' 代进去，只留 v'：

$$v' = v - (h\,c\,/\,m)\,v' - (h\,k\,/\,m)\,x - (h^2\,k\,/\,m)\,v'$$

用 $k = m\,\omega^2$，$c = 2\,\zeta\,m\,\omega$，质量被消掉：

$$v'\,(\,1 + 2\,\zeta\,h\,\omega + h^2\,\omega^2\,) = v - h\,\omega^2\,x$$
$$v' = (\,v - h\,\omega^2\,x\,)\,/\,(\,1 + 2\,\zeta\,h\,\omega + (h\,\omega)^2\,)$$

定义和源码相同的中间量：

$$a1 = 2\,\zeta + h\,\omega$$
$$a2 = h\,\omega \cdot a1 = 2\,\zeta\,h\,\omega + (h\,\omega)^2$$
$$\text{所以 } 1 + a2 = 1 + 2\,\zeta\,h\,\omega + (h\,\omega)^2$$
$$v' = (\,v - h\,\omega^2\,x\,)\,/\,(1 + a2)$$

这一步的冲量是动量变化 $\lambda = m\,(v' - v)$。它就是隐式弹簧“这一步该有的总冲量”。

5.2 三个系数

$$\text{biasRate} = \omega \;/\; a1$$
$$\text{massScale} = a2 \;/\; (1 + a2)$$
$$\text{impulseScale} = 1 \;/\; (1 + a2)$$

源码写成 a3 = 1/(1+a2)，massScale = a2·a3，impulseScale = a3。请验证两件事：

- massScale + impulseScale = 1。永远成立。
- massScale · biasRate = [a2/(1+a2)] · [ω/a1]。而 a2/a1 = h ω，所以乘出来是 h ω^2 / (1+a2)。这正好是上面 v′ 公式里 x 那一项的系数。

三个极端，源码注释也写了：

情况	biasRate	massScale	impulseScale	手感
ζ = 0	1/h	$(h\omega)^2/(1+(h\omega)^2)$	$1/(1+(h\omega)^2)$	无阻尼弹簧
ω → 很大	→ 1/h	→ 1	→ 0	退回硬约束，一步想把误差消掉
hertz = 0	0	0	0	这根约束本步不出力

5.3 写进 PGS 的那一行

v3 接触求解（略去夹紧）是：

$$\Delta\lambda = -\,\text{massScale} \cdot m_{\text{eff}} \cdot (\,\text{vn} + \text{biasRate} \cdot s\,) - \text{impulseScale} \cdot \lambda_{\text{旧}}$$

vn 是**已经包含 $\lambda_{\text{旧}}$ 效果**的当前法向相对速度。固定点是 Δλ = 0 并且 vn = $v_{\text{外力}}$ + λ/m_{eff}。把这两式代进去，得到：

$$v' = \text{impulseScale} \cdot v_{\text{外力}} - \text{massScale} \cdot \text{biasRate} \cdot s$$
$$= v_{\text{外力}} \;/\; (1+a2) - (h\,\omega^2 \;/\; (1+a2))\; s$$

和上一节的隐式欧拉是同一个式子。所以：顺序冲量如果迭代到收敛，这一根软约束的结果就是隐式弹簧走一步，不会像显式弹簧那样随硬度爆炸。没收敛时只是这个结果的近似，这和硬约束没迭代够是同一类误差。

impulseScale · $\lambda_{\text{旧}}$ 这一项不是摩擦，也不是阻尼的全部。它的作用是：λ 已经在速度里了，不能每次迭代都把整份弹簧冲量再加一遍。因为 massScale + impulseScale = 1，重复迭代会停在隐式解，而不是越加越大。

> **例子 F：一个质量，一步软约束，手算并和隐式公式对上。**
> m_{eff} = 2。间隙 s = −0.02（嵌进 2 厘米）。碰前速度 v = −1（还在往里）。子步 h = 1/240 秒。频率 30 Hz，ζ = 1。
> ω = 2π×30 ≈ 188.496
> a1 = 2×1 + (1/240)×188.496 ≈ 2.7854
> a2 = (1/240)×188.496×2.7854 ≈ 2.1876
> biasRate = 188.496 / 2.7854 ≈ 67.673
> massScale ≈ 0.6863，impulseScale ≈ 0.3137
> λ 从 0 开始，vn 就是 −1：
> Δλ = −0.6863×2×(−1 + 67.673×(−0.02)) − 0
> 先算括号：67.673×0.02 = 1.3535，所以 −1 − 1.3535 = −2.3535
> Δλ = −1.3726 × (−2.3535) ≈ 3.230
> 新速度 = −1 + 3.230/2 ≈ 0.615
> 直接套隐式公式：$v' = (-1 - (1/240)\times 188.496^2\times(-0.02)) / (1+2.1876) \approx 0.615$。两个结果相同。一次迭代就到了，因为这里只有一个约束。有第二个约束抢这个物体时，要多轮或靠子步才能靠近这个数。

接触上还有一道保险。重叠很大时，biasRate·s 会要求极高的分离速度，轻物体会被弹飞。v3 把软偏置夹到不超过 contactSpeed：

$$\text{速度偏置} = \max(\ \text{massScale} \cdot \text{biasRate} \cdot s\ ,\ -\text{contactSpeed}\ ) \qquad \text{当 } s \leqslant 0$$

s 是负数，max 会挑“没那么负”的那个，也就是限制推开有多猛。

5.4 频率不能比步子更高

奈奎斯特：一个波至少一个周期采两个点。半隐式弹簧更保守，一个周期最好 4 步以上。v3 在准备接触时把频率再夹一道：

$$\text{实际使用的频率} \leqslant 0.125 \ /\ h = 1\ /\ (8\ h)$$

4 个子步、整帧 1/60 时，h = 1/240，上限正好是 30 Hz。所以你把接触频率写成 100 Hz，在这个子步长度下仍然会被夹成 30 Hz。想要更硬的接触，先加子步，再提高频率。这就是“子步让软约束看起来更硬”的原因：h 变小，允许的 ω 变大，massScale 更接近 1。

另外，两个动态物体质量差很大时，源码会把频率和阻尼再乘一个 ratio，ratio 是较小逆质量与较大逆质量之比，但不低于 0.5。轻的那一侧如果用和重物一样硬的弹簧，顺序冲量很难在一轮里传过去。这是稳定性修正，不是新的物理定律。

6. Soft Step：v3 一个时间步里到底按什么顺序算

用户调用的是 `b2World_Step(world, timeStep, subStepCount)`。建议从 4 个子步试起。接触点**整帧只算一次**，事件也是整帧的。子步内部不再做宽相和窄相。

源码里两个常数是写死的：`ITERATIONS = 1`，`RELAX_ITERATIONS = 1`。也就是说，每个子步不是“把 8 次迭代再做一遍”，而是下面这一小圈，圈数等于子步数。

6.1 整帧开始时做一次的事

1. 宽相、窄相，得到接触流形：法线、1 或 2 个点、每个点的间隙。
2. 把锚点存成物体局部坐标。子步里物体转了、移了，用局部锚点还原世界里的 r，而不是重新跑 SAT。
3. 记下 baseSeparation：准备那一刻的间隙，再减去“当前 r 已经贡献的那一段”，这样后面只加**本帧位移**。
4. 算 K 和 m_{eff}，算 b2MakeSoft。
5. 持久岛：新接触边立刻并进岛，消失的边可以晚一点再拆岛。静态物体不进岛。整岛能量很低才一起睡。

6.2 每一个子步

1. **把力积进速度。** 重力和你施加的力按子步均分。所以“每帧设一次速度”只影响第一个子步，后面的子步会按力重新积分。想完全自己控制物体，用运动学刚体。
2. **热启动。** 把当前累积冲量先打进速度。
3. **带偏置的求解，1 轮。** useBias = true。接触在这里只做法向：重叠用软约束，尚未重叠但在逼近时用推测偏置。这一轮**不做摩擦**。
4. **用这个速度积分位置**（实际存的是相对帧初的 deltaPosition，见第 7 章）。
5. **松弛，1 轮。** useBias = false。法向不再加弹簧偏置，把上一步为了推开重叠而多出来的速度吃掉。同一轮里才做摩擦和滚动阻力。松弛改速度和冲量，**不再积分位置**。位置保留“被推开之后”的结果，速度回到更不带弹簧劲的状态。

所有子步结束之后，才做一次恢复系数。条件大致是：碰前法向速度足够负，而且这个点在整个子步过程里确实产生过法向冲量（totalNormalImpulse ≠ 0）。推测接触如果最后没撞上，不会被当成落地反弹。恢复冲量是：

$$\Delta j = -\, m_{eff}\,(\, vn + e \cdot v_{碰前}\,)$$

然后夹紧累积法向冲量，仍然不能小于 0。注释里写：两个接触点时，多迭代恢复会更准，但默认不做，成本不值得。

6.3 子步里间隙怎么更新

接触点不重算。两个物体本帧的平移差是 dp，本帧的微小旋转是 dqA、dqB。锚点局部向量转回去：

$$ds = dp + R(dqB)\, rB - R(dqA)\, rA$$
$$s = baseSeparation + ds \cdot n$$

s > 0：还没碰上。推测偏置 velocityBias = s / h，并且 massScale = 1、impulseScale = 0。单约束时这一下会把 vn 修成 −s/h，也就是“按当前步长，刚好在这一小步结束时到达表面”，而不是直接穿过

去。
s ⩽ 0：已经重叠，走软约束那一行。

> **例子 G：还没碰上，但再走一小步会穿墙。**
> h = 1/240，间隙 s = +0.02 m，法向速度 vn = −6（在靠近）。m_{eff} = 1。
> 推测分支：Δj = −1 × (−6 + 0.02×240) = −1 × (−6 + 4.8) = 1.2。
> vn′ = −6 + 1.2 = −4.8。而 −s/h = −4.8。正好。
> 这一小步的位移大约是 vn′·h = −0.02，把 2 厘米的缝走完，到达表面，而不是以 6 m/s 穿进墙里。它还不负责反弹。反弹是整帧最后的恢复趟，而且只有这个点后来真的产生了推开冲量才做。这就是 v3 用推测接触代替 TOI 子步的原因：不用在一帧中间把碰撞检测重跑很多次，高速物体用“提前一点点减速去够那个缝”。缝太大、速度又极大时仍可能不够，特别快的物体还是要靠连续检测的开关，但不再用 2.4 那种串行的撞击时刻把整个世界切开。

6.4 松弛在数字上做什么

带偏置的那一轮，为了填 s，会把 vn 推到一个偏正的值，λ 也偏大。位置已经按这个速度挪开了。松弛轮令 bias = 0、massScale = 1、impulseScale = 0，相当于再做一次“只想要 vn 不要再穿透”的硬修正，把多余的正向速度和多余的 λ 收回来。位置不跟着这次速度再走。所以你看到的是：几何上被挤出来了，动能没有保留那一下弹簧的弹跳。

摩擦放在松弛轮，是因为库仑上限要用到法向冲量。法向先在偏置轮里建立，松弛轮再更新法向并立刻夹摩擦。滚动阻力同理，上限正比于本接触的总法向冲量，只改角速度：

$$\Delta\lambda_{roll} = -\ \text{rollingMass} \cdot (\omega B - \omega A)$$
$$|\lambda_{roll}| \leqslant \text{rollingResistance} \cdot (\text{两个接触点的法向冲量之和})$$

rollingMass = 1 / (1/IA + 1/IB)，和法向的 m_{eff} 是同一思想，只是约束的是相对角速度。

7. 数值稳定：具体会坏在哪，v3 怎么避

7.1 迭代残差和子步

例子 B 已经展示：两个质量相同的约束，残差每轮减半。例子 D 展示：质量比 10 比 1、每帧只做两轮时，残差大到箱子持续加速下陷。子步把重力拆小，每小步的速度增量更小，热启动在一帧里传递多次。Solver2D 的结论是：同样的约束遍历次数下，子步通常比“大步长 + 多次迭代”更稳，因为约束在大旋转下是非线性的，步长越小，线性化（雅可比在这一步里当常数）越接近真的。

v3 的默认圈是每个子步 1 次求解 + 1 次松弛。你要更稳，增加 subStepCount，而不是找一个已经不存在的 velocityIterations。

7.2 不要用显式弹簧硬去顶接触

接触的硬度来自 massScale 接近 1 的软约束，它的稳定性继承自隐式欧拉。把接触频率调到远高于 1/(8h) 不会更硬，会被夹住。把 ζ 调成 0，重叠会振荡。接触上常用较大的阻尼比，让它更像“很快贴住”

而不是“弹簧床”。

7.3 远离原点时不要用两个大数相减

Solver2D 里 Erin 发现 XPBD 这种写法在离原点几十公里处会漂：

$$v = (position - position0) / h$$

position 和 position0 都是 30000 米量级，相减之后只剩下厘米级的运动，float32 的有效数字不够，物体会突然停住。修法是求解器全程只累积 deltaPosition，速度用增量除以 h，绝对坐标等到这一帧结束再加回去。碰撞点也存在物体局部坐标里，不存世界坐标大数。子步更新间隙时，dp 是本帧位移增量，不是两个世界坐标相减。物体越小、离原点越远，这个问题越早出现。调试时在画面上放一根 1 米的比例尺，能看出你是不是已经离原点太远。

7.4 力、冲量、运动学

子步会反复积分力。你如果在帧初把速度设成一个值，第一个子步之后求解器会改它，后面的子步不会自动恢复你设的值。持续控制用 ApplyForce。一次性打击用冲量，但要知道它主要影响接下来的子步，不会在每个子步被重放。平台、电梯、必须严格按脚本走的物体用 kinematic。

7.5 睡眠必须整岛

只让一个箱子睡觉、它上面的箱子还醒着，连接它们的接触不再被求解，重叠和关节错位会立刻出现。醒来也要沿着接触和关节传遍整岛。静态地面不放进岛，否则站在同一块地面上的所有东西都变成一个岛，睡眠和并行都没有意义。

7.6 多线程不要弄乱顺序

顺序冲量的结果和约束顺序有关。例子 B 如果先做 H 再做 G，中间表格的数字会不同，最后收敛到的极限相同，但只做 1 轮时结果不同。多线程如果用原子自增抢接触数组下标，谁的核跑得快，顺序就变，画面会每次数不一样。v3 让每个工人在自己的位图上做标记，主线程按位或，再按位的固定顺序处理。这样线程数变化也不会改变接触顺序。

跨平台时还要避开两件事：编译器的 fast-math，以及 FMA（a+b×c 合成一条指令，舍入和分开算不同，而且不是每颗 CPU 都有）。sqrt 可以用硬件指令。atan2 各家标准库结果不同，Box2D 用自己的近似。sin/cos 在他测过的平台上碰巧一致，但旋转构造函数仍走自带实现。回滚（联网预测）需要把岛、热启动冲量、接触身份全部存档，v3 没做这件事。

8. 摩擦、恢复、图着色：实现时和公式接在哪里

8.1 摩擦

切线 t 由 n 转 90° 得到，源码用右垂直。切向相对速度 vt 还要减掉传送带的 tangentSpeed。松弛轮里：

$$\Delta j_t = -\, m_{eff,t} \cdot vt$$
$$|\lambda_t| \leqslant \mu \cdot \lambda_n$$

夹的是累积切向冲量。法向冲量为 0 时，摩擦上限是 0，悬空擦过不会产生摩擦力。μ 是两边材料混合之后的摩擦系数。

> **例子 H：法向已经有 $\lambda_n = 7.2$，想刹住的切向冲量是 −4.32，μ = 0.3。**
> 上限 $\mu\,\lambda_n = 2.16$。$4.32 > 2.16$，所以实际 $\lambda_t = -2.16$，滑动还在，只是变慢。这是动摩擦。
> 若 μ = 1，上限 $7.2 > 4.32$，λ_t 可以取 −4.32，切向相对速度被修到 0。这是静摩擦成功。v3 仍然用一个 μ，不分成两个系数。

8.2 恢复为什么单独一趟

如果在每个子步的软约束里再乘 (1+e)，重叠修正和反弹会加在同一个冲量上，轻碰也会跳，堆叠不容易睡着。v3 把“填缝”交给软约束和推测偏置，把“碰前有多快，碰后弹回多少”留到最后一趟，并且速度不够负（低于阈值）就跳过。落地的箱子不会永远微微蹦。

8.3 图着色和 SIMD 不改变公式

同一刚体上的两个接触不能同时写这个刚体的速度，否则数据竞争，结果还和串行 PGS 不同。图着色把接触分成若干色，同色的接触不共享刚体，于是这一色可以并行，也可以装进 AVX 的多条通道里一起算**同一条**冲量公式。溢出色（overflow）放那些涂不进颜色的动态–动态接触，在每一轮里先于各颜色、单线程求解，优先级较低。

所以读 `contact_solver.c` 时可以分成两层：标量的 overflow 版本把公式写直白；wide 版本是同一公式的 SIMD。调试数值时看标量版本即可。

9. 把一条接触从准备算到速度改变

下面按 v3 子步里的真实分支，把符号走完。设一个子步，h = 1/240。箱子角撞静止的墙，数据沿用有效质量那一问：m = 2，I = 4/3，r×n = −1，K = 1.25，$m_{eff} = 0.8$。这一问先不考虑旋转以外的切线，ζ = 1，频率被夹在 30 Hz。软系数就是例子 F 的那一组：massScale = 0.6863，impulseScale = 0.3137，biasRate = 67.673。

情形 1：已经重叠 s = −0.01，vn = −2，λ = 0。偏置轮。
速度偏置 = massScale × biasRate × s = 0.6863 × 67.673 × (−0.01) ≈ −0.464。
Δλ = −0.8 × (0.6863×(−2) + (−0.464)) − 0
括号内：−1.3726 − 0.464 = −1.8366
Δλ = −0.8 × (−1.8366) ≈ 1.469
质心沿法线的速度变化 = −(Δλ)/m （墙在 +n 一侧、箱子是 A 时要看谁是 A/B；若箱子是被墙推开的那一侧，速度沿离开墙的方向增加 Δλ/m = 0.735）。
角速度变化 = ± (1/I) × (r×n) × Δλ = ±0.75×(−1)×1.469，约 ∓1.10 rad/s。
然后积分位置。松弛轮再用新的 vn、且不再加 −0.464 那一项，把偏大的速度收一收。

情形 2：还分开 s = +0.015，vn = −5。推测轮，不是软约束。
velocityBias = 0.015 × 240 = 3.6。massScale 被设成 1，impulseScale 被设成 0。
Δλ = −0.8 × (−5 + 3.6) = −0.8 × (−1.4) = 1.12。
vn 的变化 = Δλ / m_{eff} = 1.12 / 0.8 = 1.4。
vn′ = −5 + 1.4 = −3.6 = −s/h。
这一子步刚好赶上表面。若下一子步 s 变成负的，才进入情形 1 的软约束。

情形 3：两个接触点的箱子躺在地上。
只有一个点时，箱子可以绕这个点转进地面。左右各一个点，每个点有自己的 λ_n、自己的 m_{eff}（因为 r 不同，叉积不同）。顺序是先点 1 再点 2，点 2 会破坏点 1，所以需要子步和热启动。块求解是把两个点的 2×2 小矩阵一次解完，2.4 用过；v3 不在热路径上做这个，靠子步缩小每次的破坏。

10. 和 2.4 对照，避免用错脑模型

	Box2D 2.4	Box2D 3 Soft Step
你调什么	velocityIterations、positionIterations	subStepCount。内部每子步 1 次求解 + 1 次松弛
重叠	主要靠 NGS 位置迭代	软约束偏置 + 推测接触。松弛去掉多余速度
弹性	混在速度迭代里	所有子步之后单独一趟
摩擦	和法向在同一迭代里先后做	放在松弛轮，此时 useBias 为假
接触点	每帧检测一次，迭代中锚点基本固定	每帧检测一次，子步用局部锚点更新 s
快物体	TOI，串行把步长切开	推测减速为主，避免那条串行瓶颈
岛	每帧 DFS 现建，用于睡眠	持久岛，位图合并，服务睡眠和并行
大岛并行	一个岛一个线程，大堆只能一个核	岛内再图着色，同色接触 SIMD / 并行
竖直叠箱	块求解很强	稍弱，用子步换整体稳定和速度

11. 读源码时的检查清单

1. `b2MakeSoft`：只看 hertz、zeta、h。先确认 massScale + impulseScale 是否为 1。
2. 准备接触：baseSeparation、局部锚点、normalMass = 1/K。频率有没有被 0.125/h 夹住。
3. 子步循环：积速度 → 热启动 → useBias 真（法向）→ 积位置 → useBias 假（法向 + 摩擦 + 滚动）。
4. s > 0 走 s/h，s ≤ 0 走软偏置，并且软偏置被 contactSpeed 夹住。
5. 夹紧的是累积 λ，加到物体上的是新旧 λ 之差。
6. 恢复在子步循环外面，看 totalNormalImpulse 和碰前速度阈值。
7. 宽 SIMD 文件和 overflow 标量文件应实现同一公式。数值对不上时以标量文件为准。
8. 行为不重复：先查接触顺序和 fast-math / FMA，再查是不是在帧初写速度却期望每个子步都保持。

12. 公式速查

$$K = 1/mA + 1/mB + (rA \times n)^2/IA + (rB \times n)^2/IB$$

$$m_{eff} = 1/K$$

硬接触、e = 0：$\Delta\lambda = -m_{eff}\, vn$，再夹到累积值 ≥ 0

软接触：$\Delta\lambda = -\text{massScale}\; m_{eff}\,(vn + \text{biasRate}\; s) - \text{impulseScale}\; \lambda$

$$\omega = 2\pi f,\quad a1 = 2\zeta + h\omega,\quad a2 = h\omega\, a1$$

$$\text{biasRate} = \omega/a1,\quad \text{massScale} = a2/(1+a2),\quad \text{impulseScale} = 1/(1+a2)$$

推测：s > 0 时 $\Delta\lambda = -m_{eff}\,(vn + s/h)$

摩擦：$|\lambda_t| \leq \mu\, \lambda_n$

子步间隙：$s = \text{baseSeparation} + ds \cdot n$

13. 建议的阅读顺序

1. 先看第 2 章例子 A 和 B，把“一个冲量”和“两个约束互相破坏”算一遍。
2. 再看例子 C，理解热启动为什么能让堆叠停住，以及为什么会多推出 0.03 这种残差。
3. 第 5 章把例子 F 和隐式欧拉对一下。对上之后，Soft Step 就不再是一组魔法系数。
4. 第 6 章对着 `solver.c` 的子步循环看，确认摩擦在松弛轮、恢复在循环外。
5. 最后用例子 G 看推测接触，用第 7 章看浮点和多线程顺序。

原始材料入口：https://box2d.org/posts/ 以及 box2d.org/files/ 下的 Erin Catto GDC PDF。实现入口：github.com/erincatto/box2d 的 src/solver.c、src/solver.h、src/contact_solver.c。若源码以后改了 ITERATIONS 或偏置夹紧方式，以源码为准，用第 11 章的清单去对。